

Magnetic Field Scanner - Project Overview

Document Version: v1.3.0 **Last Updated:** September 14, 2026 **Firmware Target:** v5.1.4 **Python Ecosystem Target:** v1.1.1

This document serves as the master source of truth for the Magnetic Field Scanner project. It details the mechanical assembly, electrical wiring, pinouts, firmware architecture, critical pitfalls to avoid in future development, and the roadmap for upcoming features.

1. Project Identity, Versioning & Documentation Rules

- **CRITICAL RULE:** All projects shall have a main Software version identifier as `v[Major].[Minor].[Patch]` and follow industry standard semver rules and practices for bumping the version at EVERY file iteration.
- **Why:** We do not reuse version numbers for each iteration. This allows us to distinctly identify in the AI context the files we are working on. The version string is defined as `FIRMWARE_VERSION` in `user_config.h`.

Documentation Strategy (The "Definition of Done"): To prevent documentation rot and excessive maintenance overhead, documentation updates are strictly tethered to the SemVer lifecycle:

- **Patch Releases (e.g., v3.1.1 -> v3.0.33):** Bug fixes and math tweaks. Do *not* update the architectural docs (`project_overview.md`, `user_manual.md`). Log these changes exclusively in the root `CHANGELOG.md`.
 - **Minor Releases (e.g., v3.0.x -> v3.1.0):** Adding new functional features (e.g., GPS integration). The feature is not considered complete until both the `CHANGELOG.md` and the relevant `docs/` Markdown files are updated to reflect the new capabilities.
 - **Major Releases (e.g., v3.x -> v4.0.0):** Massive architectural overhauls (e.g., migrating from I2C to CAN bus). Triggers a comprehensive review and rewrite of all files in the `docs/` directory.
-

2. Hardware Architecture & Mechanical Setup

- **The Wand:** Constructed from a rigid 1" fiberglass outer rod with a 3/4" ID.
- **Main Controller (The Handle):** Waveshare ESP32-S3-Touch-LCD (4.3 / 3.49 v2) with an integrated screen, battery ADC, IMU (QMI8658, addr 0x6B), and RTC (PCF85063, addr 0x51). This unit sits at the top of the wand acting as the primary handle and UI display. A custom wiring harness runs down from this unit into the main fiberglass tube.

- **The Sensor Array:** The wand acts as a **Magnetic Gradiometer**. It currently utilizes two PNI RM3100 magnetometer breakout boards ("Tip" and "Reference").
- The boards are mounted to a PLA mechanical carrier that slides directly into the 3/4" ID of the fiberglass tube.
- **Carrier Rigidity:** To ensure absolute rigidity between the Tip and Reference sensors, the PLA board carriers are physically joined to each other by a solid 1/4" internal fiberglass rod.
- **Sensor Orientation:** The boards were recently flipped 180 degrees (upside down) in the assembly.
- **Physical Misalignment:** There is a permanent, measured physical flex/bend in the carrier assembly of roughly ~9.4 to ~9.8 degrees. The firmware mathematically eliminates this using the Kabsch alignment algorithm.

Mechanical Diagram

(Assuming visual diagram exists elsewhere)

Sensor Coordinate Systems & Axis Alignment (Phase 7 Discovery)

Because the BMI270 IMU and the dual RM3100 Magnetometers are mounted to the same rigid Wand structure, they must speak the exact same 3D spatial language for sensor fusion (Madgwick 9-DOF) to work. A live logging "Alignment Dance" revealed the exact physical orientation of the chips on the Veroboard.

The IMU (BMI270) Coordinate System (Right-Handed):

- +X Axis: Points **Forward** (Towards the sensor tip)
- +Y Axis: Points **Left**
- +Z Axis: Points **Up** (Out of the LCD Screen)

The Magnetometer (RM3100) Coordinate System (Left-Handed): Due to the physical mounting of the breakout boards inside the wand, the RM3100 acts as a Left-Handed coordinate system relative to the IMU.

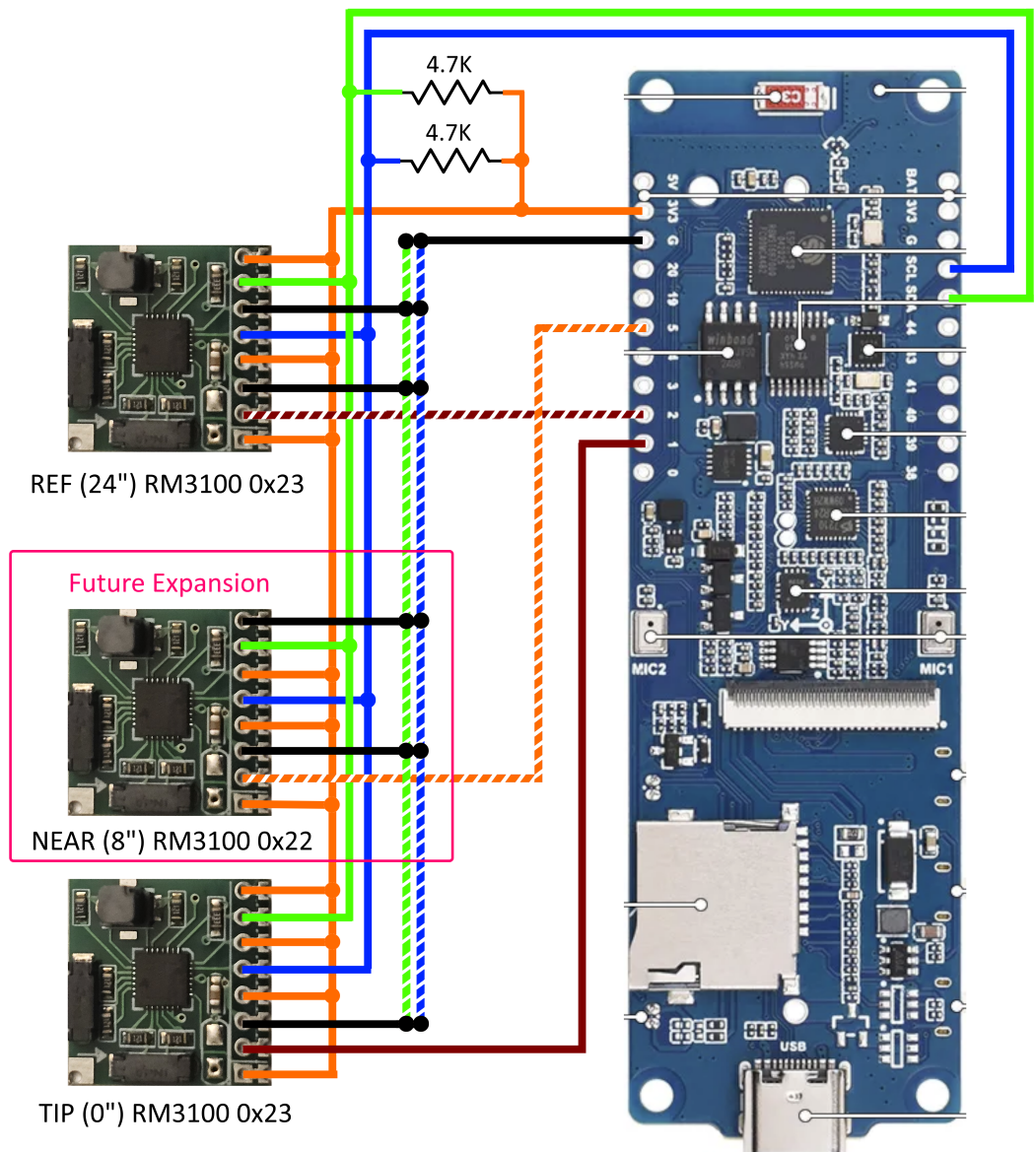
- +X Axis: Points **Left**
- +Y Axis: Points **Forward**
- +Z Axis: Points **Down** (Into the ground)

The Translation Key: To safely feed the Magnetometer data into the Right-Handed 9-DOF Madgwick algorithm, the software applies the following mathematical mapping to the RM3100 (efX_cal, refY_cal, refZ_cal) values: `c float mag_x = (float)ref.y; // Maps RM3100 Forward to IMU Forward float`

```
mag_y = -(float)ref.x; // Maps RM3100 Right to IMU Left float mag_z = -(float)ref.z; // Maps  
RM3100 Down to IMU Up This permanently locks the Quaternions to True Magnetic North for accurate  
post-analysis mapping. (Please place your mechanical drawing in a folder named docs in the project  
root and name it mechanical_drawing.png or update this link to point to it).
```

3. Wiring & Pinout Assignments

Magnetic Field Scanner Wiring Diagram
2026-09-14



Waveshare ESP32-S3 Touch LCD 3.49 v2 (PCBA v1.1)

Pin	Function	CABLE (Note)
3V3	3V3	ORG
GND	GND	BLK, GRN/WHT, BLU/WHT
IO47	SDA	GRN (4.7K Pullup to 3V3)
IO48	SCL	BLU (4.7K Pullup to 3V3)
IO01	DRDY_TIP	BRN
IO02	DRDY_REF	BRN/WHT
IO05	DRDY_NEAR	ORG/WHT

CRITICAL HARDWARE NOTE: I2C Pull-Up Resistors

- **The RM3100 breakout boards DO NOT have built-in I2C pull-up resistors.**
- **The Fix:** We permanently installed **4.7K pull-up resistors** in the cable harness directly at the Waveshare connection end (SDA/SCL to 3.3V).
- **The Result:** Combined with the Waveshare's internal pull-ups, this provides a "stiff" ~2K to ~3K ohm equivalent resistance, which is perfect for fighting the heavy capacitance of the 3-foot cable run down the wand.

CAT6 Wire Harness & Color Coding

The internal harness from the Waveshare board to the sensor array utilizes a standard CAT6 twisted-pair cable. It perfectly allocates all 8 wires to support three RM3100 sensors while maximizing electrical integrity over the 48" run.

For the full visual diagram, reference the spreadsheet: docs/electrical/Cable_Color_Code.xlsx.

Component / Function	ESP32 Pin	CAT6 Color	Twisted Pair Strategy (Signal Integrity)
3.3V Power	3V3	Orange	Power & Sync: Twisted with DRDY_NEAR. DC power acts as an AC ground, shielding the digital pulse.
DRDY_NEAR	IO05	Orange/White	<i>(Future 3rd Sensor)</i>
SDA (Data)	IO47	Green	I2C Data Shield: Twisted directly with GND to prevent capacitive crosstalk against SCL.
GND	GND	Green/White	

Component / Function	ESP32 Pin	CAT6 Color	Twisted Pair Strategy (Signal Integrity)
SCL (Clock)	IO48	Blue	I2C Clock Shield: Twisted directly with GND to prevent signal degradation.
GND	GND	Blue/White	
DRDY_TIP	IO01	Brown	Digital Sync: Hardware data-ready interrupt signals.
DRDY_REF	IO02	Brown/White	

I2C Architecture & Device Addresses

The wand utilizes two completely separate hardware I2C buses to isolate the sensitive long-wire sensors from the noisy high-speed screen components.

1. I2C0 (External Sensor Bus)

- **Purpose:** Drives the long wire harness running down the 3-foot fiberglass tube to the RM3100 sensors.
- **Speed:** Hardcoded to **100kHz** to fight cable capacitance.
- **Pins:** SCL: 48, SDA: 47
- **Devices on Bus:**
 - **RM3100 Tip:** 0x23 (DRDY: GPIO 1)
 - **RM3100 Ref:** 0x21 (DRDY: GPIO 2)
 - **RM3100 Mid (Future):** 0x22 (DRDY: GPIO 5) *(Reserved for planned 3rd sensor)*
- **CRITICAL CONFLICT HISTORY (Address 0x20):** Initially, an RM3100 was set to address 0x20. This caused a total system failure because the Waveshare board has an undocumented internal I/O Expander hardcoded to 0x20. **Never use 0x20 on the sensor bus**, even though the buses are logically separated; we mapped the RM3100s to 0x21, 0x22, and 0x23 to safely avoid any internal

hardware routing conflicts.

2. I2C1 (Internal Waveshare Bus)

- **Purpose:** Local communication for the Waveshare board's integrated SMD components.
- **Pins:** SCL: 18, SDA: 17
- **Devices on Bus:**
- **Capacitive Touch Controller:** 0x3B
- **RTC (PCF85063):** 0x51
- **IMU (QMI8658):** 0x6B
- **I/O Expander:** 0x20 (*The source of the conflict mentioned above*)

Other Master Pinouts (Defined in user_config.h)

Component	Pin (GPIO)	Notes / Details
SPI (SD Card)	CS: 38, MOSI: 39, MISO: 40, CLK: 41	Used by <code>wifi_logger.cpp</code> to save <code>calibration.csv</code> and logs.
GPS Module	TX: 43, RX: 44	<i>Planned Future State.</i>

4. Critical Engineering Decisions & Pitfalls to Avoid

A. I2C Bus Speed & Time Constants

- **The Mistake:** Attempting to run the hardware I2C bus at 400kHz ("Fast Mode") over a 60-inch unshielded CAT6 cable. The physical capacitance caused mushy square waves. The ESP32 missed ACK bits, resulting in 0,0,0 readings, massive garbage spikes, and frequent "Sensor Lockup" panics. 100kHz was extremely stable but too slow for high-speed logging.
- **The Fix:** The I2C speed (`scl_speed_hz`) in `i2c_bsp.c` is hardcoded to 200,000 Hz. This is the "Goldilocks" zone that perfectly balances RC time constant degradation against the bandwidth needed to poll both sensors at 150Hz.

B. EMI "Meteors", Slew Rate, & Absolute Filters

- **The Problem:** Even at 200kHz, the long I2C wires act as antennas. Microscopic bit-flips (EMI glitches) occasionally jump the raw reading by 2,000 to 5,000 counts in a single frame, causing the audio UI to emit a terrifying shriek ("cosmic meteors").
- **Slew Rate Filter:** If any single axis on either sensor jumps by more than **800 counts** in a single frame, it is mathematically deemed physical EMI and the frame is dropped.
- **Absolute Magnitude Glitch Filter (MFS_MAX_GLITCH_MAGNITUDE):** Protects against catastrophic I2C garbage data (e.g. 140,000 counts). *CRITICAL PITFALL:* This threshold **MUST** be set >250,000. At CC=3200, the raw sensor gain is so massive that perfectly valid measurements hit ~120,000 LSB. Setting this threshold too low (e.g. 50,000) will cause the software to silently reject every single CC=3200 frame, freezing the display and log entirely!

C. Watchdog Timer (TWDT) Panics

- **The Problem:** Processing a 3,000-line `calibration.csv` file from the SD card using `String` operations in `wifi_logger.cpp` takes more than 5 seconds. The ESP32 FreeRTOS Task Watchdog Timer assumed the system was frozen and triggered a hard panic reboot right at 99% completion.
- **The Fix:** Ensure heavy processing loops (like file parsing) contain a `vTaskDelay(pdMS_TO_TICKS(10));` every 100 iterations to "pet the watchdog" and yield to the OS.

D. Sensor Lockup Logic & Hardware Watchdog

- **The Problem:** The I2C bus will occasionally hang due to environmental EMI. Previously, a 5000ms I2C driver timeout would starve the `IDLE0` task and trigger a fatal ESP32 Task Watchdog Panic reboot.
- **The Fix:** The I2C driver timeouts (`i2c_data_pdMS_TICKS`) are clamped to 50ms. If the bus hangs, it fails gracefully. The main `SensorTask` implements a robust software watchdog that resets the RM3100s via `initRM3100()` under three conditions:
 1. **Stuck Data:** If the sensor returns exactly 0,0,0 for 10 consecutive frames (I2C read failure). *CRITICAL PITFALL:* Do not trigger on identical valid values (e.g. `tip.x == last_tip.x`)! The RM3100 noise floor is so low that a perfectly still sensor will trigger false-positive lockups!
 2. **DRDY Timeout:** If the `DRDY` event group times out after **1000ms**. *CRITICAL PITFALL:* CC=3200 physics takes ~480ms to measure. If the timeout is set to 500ms, the ESP32 will panic and reset the sensor *before it finishes*, causing an infinite freeze loop!
 3. **Blind Recovery I2C Reads:** When recovering from a timeout, **NEVER** issue a blind `i2c_read_buff` to the RM3100 unless you have verified via `digitalRead()` that the `DRDY` pin is physically stuck HIGH. Reading a busy RM3100 will crash its internal silicon and permanently hang the I2C bus!

D2. RM3100 TMRC Internal Glitching

- **The Physics:** `TMRC` defines the update rate in Continuous Measurement Mode. The datasheet claims that if `TMRC` is set faster than the Cycle Count allows, it will "automatically override" and run at max speed.
- **The Reality:** While true, constantly bombarding the RM3100's internal state machine with 600Hz `TMRC` ticks while it is busy calculating a slow 400 CC measurement causes random silicon glitches, resulting in the sensor missing beats and dropping `DRDY` (triggering the watchdog).
- **The Fix:** `initRM3100()` must use a rigorous mapping table to ensure the `TMRC` value is always safely **slower** than the physical Cycle Count math execution time.

E. Universal Cycle Count Scaling (v5.x.x)

- **The Physics:** When the user changes the Cycle Count (CC) on the fly, the RM3100's raw output scales non-linearly due to Zero-Field Offsets (ZFO).
- **The Code (Pre-Normalization):** To eliminate drift and scaling errors, the v5.1.4+ architecture intercepts raw LSB counts and immediately normalizes them to physical nanoTeslas (nT) using the exact cc gain scalar *before* any calibration is applied.
- **Universal Matrices:** Because the Kabsch calibration algorithm now operates entirely in the physical nT domain, the resulting Hard and Soft Iron matrices are mathematically dimensionless. You can calibrate the wand at 400 CC, and perfectly seamlessly jump to 3200 CC without re-calibrating or scaling the matrices!

F. Auto-Tare vs Manual Tare

- **Manual Tare:** Memorizes the current environmental gradient shadow (e.g., standing near a car) and subtracts it from all future readings. Should only be performed in a magnetically "clean" area.
- **Auto-Tare:** Implements a low-pass filter (multiplier 0.005) that slowly and invisibly pulls the baseline back to zero over time. Crucially, it **only engages if the gradient jumps by less than 150 nT**. This eats away slow temperature/geology drift while completely ignoring the sharp spikes of a buried utility pipe!

G. Windows CLI Path Handling (Python)

- **The Problem:** When dragging and dropping files into Windows PowerShell or Command Prompt to pass arguments to Python scripts, Windows automatically wraps the filepath in literal double-quotes (e.g., "C:\path\to\file.csv"). Python's `argparse` absorbs these literal quotes as part of the string, causing `os.path.exists()` to fail and crash the script.
- **The Fix:** All Python companion scripts in this project universally sanitize `argparse` inputs using `os.path.normpath(args.input.strip('\'))` before interacting with the file system. This strips accidental terminal quotes and normalizes slashes, bulletproofing the CLI against Windows pathing

quirks.

5. Current State of the Project

- **Core Functionality:** The Gradiometer math (`Tip - Ref`) is fully operational and extremely sensitive.
- **On-Device Calibration:** Functional. `matrix_math.h` performs a 9x9 Jacobi eigenvalue solver and 3D ellipsoid fit using only local stack memory. It applies Kabsch alignment to precisely orient the Tip sensor onto the Reference sensor's frame, nullifying the 9-degree mechanical flex.
- **Data Logging:** SD card saving is stable. Python companion scripts (`calibrate_wand.py` and `plot_pins.py`) can ingest `_log.csv` and `_calibration.csv` files to visualize the exact sweeping motions using IMU Gyroscope Z-axis tracking.
- **Audio & UI:** LVGL runs successfully on Core 1 while the bitbanging/I2C sensor tasks run on Core 0. (Note: `lv_conf.h` must remain in the root directory due to library constraints).

6. Companion Software Ecosystem (Python)

The repository includes a suite of offline Python processing scripts located in the `scripts/` directory. All official scripts adhere to SemVer versioning, contain industry-standard header documentation, and utilize `argparse` for robust command-line `--help` functionality.

- **`calibrate_wand.py`:** Ingests raw `calibration.csv` datasets, computes the 3x3 Soft-Iron matrices (W) and Hard-Iron center offsets (V), applies Kabsch rotational alignment, and outputs a drop-in `calibration.json` file.
- **`generate_plots.py`:** Specialized utility for visualizing 3D ellipsoid point clouds to verify the mathematical perfection of the sphere-fitting algorithms.
- **`analyze_log.py` (Formerly `plot_pins.py`):** Ingests live field operation logs (`_log.csv`) and generates a synchronized 3-pane matplotlib dashboard showing Gradient Magnitude, Audio Frequency response, and IMU Gyroscope Z-axis (Yaw) to visually correlate physical sweeping motions with underground utility strikes.

7. Planned Future State (Roadmap)

1. **Third Sensor Integration (Near):** * A third RM3100 sensor will be mounted at 8 inches from the TIP. * I2C Address: `0x22` * DRDY Pin: `GPIO_5` * Purpose: To allow quadratic gradient tracking and depth-estimation of buried targets.

2. **GPS Module Integration:** * Serial unit to be added to pins I043 (TX) and I044 (RX). * Purpose: To embed geospatial coordinates into the SD log files, allowing post-processing software to map buried utilities directly onto a satellite view.

8. Sensor Fusion (Madgwick 9-DOF) & IMU Mathematics

The firmware utilizes a hyper-optimized implementation of Sebastian Madgwick's AHRS (Attitude and Heading Reference System) to fuse the BMI270 (Accel/Gyro) and RM3100 (Magnetometer) into a buttery-smooth 3D quaternion vector. This allows the wand to project the gradient magnitude perfectly onto the flat horizontal plane (Radar Dot) and ignore false signals generated by tilting the wand.

A. Coordinate System Alignment (NED)

Because the RM3100 and the IMU are physically mounted in different orientations, they are mathematically mapped into a strict **North-East-Down (NED)** coordinate system before fusion.

- **Accelerometer:** (-accX, accY, accZ)
- **Magnetometer (Ref):** (refY, refX, refZ)
- **Gyroscope:** (gyrX, -gyrY, -gyrZ)

B. Display Enclosure Mechanical Tilt (imu_rotation_deg)

The BMI270 IMU is located inside the display head, which is physically pitched UP by 60 degrees relative to the straight wand tube.

- To prevent the math from permanently thinking the wand is aiming at the sky, a mathematical rotation is applied to un-tilt the IMU readings.
- **Order of Operations (CRITICAL):** 1. Gyro Hardware FOC (Zero-Rate Offset) is subtracted from the raw silicon axes. 2. The NED axes are mathematically rotated by -cal_config.imu_rotation_deg (Negative to *un-rotate* the positive physical pitch). 3. The FOC-corrected, rotated axes are converted to Radians and fed into Madgwick.

C. Gyroscope FOC & Dynamic Auto-Zero (Tare)

MEMS Gyroscopes (like the BMI270) suffer from thermal drift and high "g-sensitivity" (where laying the wand on its side causes gravity to pull the silicon mass differently, altering the zero-rate offset by up to 4 dps). A static calibration file is completely insufficient to cancel this.

- **The Solution:** The SensorFusion class implements a real-time **Dynamic Auto-Zero** algorithm.
- It maintains a silent 1-second rolling buffer of the Accelerometer and Gyroscope.

- If the variance of the Accelerometer drops below 0.001g (meaning the wand was set down on a desk/ground), it instantly recalculates the Gyroscope FOC from the buffer and updates the RAM seamlessly.
- This completely eliminates Elevation and Azimuth drift regardless of temperature or resting orientation.

D. The Madgwick Math Quirks

- **Degrees vs Radians:** The C implementation of MadgwickAHRS strictly expects the Gyroscope input in **Radians per second**. Feeding it Degrees per second will amplify gyroscope noise by 57.3x, causing violent continuous wobble (hunting for equilibrium) as the filter wrestles with the Accelerometer.
- **Dynamic Time Delta (dt):** Rather than relying on a static `sampleFreq` define (which creates time-dilation distortion when hardware cycle counts alter update rates from 150Hz down to 9Hz), `SensorFusion` dynamically measures elapsed microseconds (`micros()`) between frames and passes `float dt` directly into `MadgwickAHRSupdate`. This guarantees rapid, rock-solid attitude convergence across all cycle count settings.

9. Firmware Architecture & Single Responsibility Principle

Following the Phase 3 structural refactor, the firmware heavily adheres to the **Single Responsibility Principle (SRP)**. Monolithic files have been rigorously decomposed to prevent cross-domain contamination, particularly between Core 0 (Sensor/SD) and Core 1 (UI/Web).

src/ Directory Breakdown:

- **sensor_fusion/:** A dedicated, stateful C++ class that handles all 3D matrix algebra, auto-tare EMA filtering, Madgwick AHRS updates, and vector geometry for the radar dot. Extracts all dense math from the main `.ino` loop.
- **data_logger/:** Solely responsible for writing CSV data to the SD card/FFat and handling file flushes.
- **settings_manager/:** Exclusively manages the saving, loading, and JSON serialization of `SystemSettings` and `CalibrationConfig`.
- **web_server/:** A pure UI/networking layer running on Core 1. It handles HTTP endpoints, file downloads, uploads, and OTA operations without directly touching hardware logic.
- **screenshot/:** Isolated LVGL screen capture logic (RGB565 to BMP conversion).
- **lvgl_port/:** Handles the physical drawing of the screen and UI layout.

- **matrix_math/**: A pure, header-only implementation of the 9x9 Gaussian elimination and 3D Kabsch alignment algorithms.
- **Hardware BSPs (i2c_bsp, imu_bsp, tc_bsp)**: Board Support Packages that isolate the direct register-level I2C commands from the main business logic.

Core Separation & Thread Safety:

- **Core 0 (Data/Hardware)**: ask_sensor_read, ask_battery_monitor, ask_audio_alert. I2C reads and fast math.
- **Core 1 (UI/Network)**: ask_display_update (LVGL), mfs_button_pwr_task, WiFi server.
- **Mutexes**: mfs_lvgl_lock() must be acquired before touching UI elements from Core 0. log_mux must be acquired before writing to logFile.

10. Critical Hardware Quirks & Engineering Notes

1. Battery Power Keep-Alive Latch (SYS_EN)

When powering on the ESP32 via the physical battery button, the hardware PMIC requires the ESP32 to quickly assert the \SYS_EN\ pin HIGH (via the TCA9554 IO Expander on I2C) to latch the power circuit on.

- **The Pitfall**: If the firmware waits for a Serial connection or performs long blocking tasks before initializing the IO Expander, the user will be forced to physically hold the power button down for several seconds. If they let go early, power is cut instantly.
- **The Solution**: The I2C bus (i2c_master_Init()) and the IO Expander (esp_io_expander_set_level(io_expander, MFS_EXIO_PIN_SYS_EN, 1)) must be executed as the absolute very first commands in \setup()\, BEFORE any \while(!Serial)\ delay loops.

2. True Polling Rates (v5.1.4 ISR Architecture)

The firmware utilizes a true zero-latency ISR architecture driven by FreeRTOS Event Groups. The sensor task sleeps at 0% CPU until the RM3100 hardware asserts the DRDY interrupts, at which point it wakes and reads instantly.

The polling rate is exclusively governed by the RM3100 TMRC hardware register, which is dynamically mapped to be safely slower than the physics of the Cycle Count (CC):

- **CC 200**: 150 Hz (0x94)
- **CC 400**: 37 Hz (0x96) - Default
- **CC 800**: 18 Hz (0x97)

- **CC 1600:** 9 Hz (0x98)
- **CC 3200:** 4.5 Hz (0x99)

Logging Decimation: To prevent SD Card SPI latency from dropping frames at 150Hz, the data logger decimates 200 CC readings by half (logging at 75Hz) while keeping the math and UI running at the full 150Hz.

3. RM3100 Sensor Physics & Limitations

Through extensive physical characterization, we have identified two distinct failure modes for the RM3100 sensors when exposed to massive magnetic anomalies:

Digital Clipping Limit (Integer Overflow)

The maximum valid magnetic field the hardware can compute is inversely proportional to the Cycle Count (CC). The RM3100 works by counting high-frequency oscillator pulses over a set number of LR cycles.

- **At low CC (e.g., 200 CC, 400 CC, 800 CC):** The measurement cycle is extremely fast. Because it is fast, the internal 24-bit counter has a massive dynamic range and will not digitally overflow until exposed to immense fields ($> 500 \mu\text{T}$).
- **At high CC (e.g., 3200 CC):** The measurement cycle stays open 16x longer. Because it counts for so long, a strong field will cause the internal register to mathematically wrap/overflow much sooner. At 3200 CC, the digital clipping limit is roughly **$\sim 88 \mu\text{T}$** (or $\sim 20 \mu\text{T}$ if perfectly aligned to a single axis).

Physical Core Saturation (The "Blind State")

If a massive magnetic anomaly (such as a neodymium rare-earth magnet) is brought within inches of the sensor, the magnetic field exceeds the physical B-H curve limit of the tiny LR inductor core.

- When this occurs, the core's magnetic permeability drops to that of air, and the inductance collapses.
- The RM3100 fails to oscillate and outputs a raw count of practically 0.
- Because the Tip sensor goes "blind" (0 nT), the Gradiometer math ($| \text{Tip} - \text{Reference} |$) will simply output exactly what the Reference sensor is reading.
- This results in the wand falsely displaying a steady-state value of **$\sim 60 \mu\text{T}$** (the Earth's background magnetic field) during a massive $> 500 \mu\text{T}$ event.
- **Important:** This is why the Open Air Rebar test (which concentrates the Earth's field but isn't a permanent magnet) is used to test the *Digital Clipping Limit*, while a Neodymium magnet forces

Physical Core Saturation.

The Gradiometer Return Path ("Destructive Null Dip")

Because the reference sensor is mounted ~24 inches behind the tip sensor on the wand handle, it sits directly inside the near-field magnetic return path of large linear dipoles (like a 36-inch rebar). While the tip sensor measures a massive positive spike as the rebar approaches, the reference sensor measures the negative return-flux pulling in the opposite direction. This opposing flux subtracts from the Earth's background field, causing the reference sensor to measure a distinct "dip" (e.g., dropping from 50 μT down to 48 μT). This proves the spatial gradiometer geometry is successfully interacting with the 3D footprint of the anomaly.